

Linear, Geometric, and Structural Transformations

José Portillo

Release date: February 15, 2026

Version: 0.4

To the memory of
Mercelena Portillo Holskin,
Eldon C. Hall,
Hugh Blair-Smith

To my family

Contact:

<https://www.linkedin.com/in/jose-portillo-643138a1/>

Contents

Preface	iv
1 Linear Transformations, Matrices, and Composition	1
2 Geometry from Linear Structure	13
3 Tensors as Multilinear Maps	29
4 Logarithms as Structure-Preserving Transformations	41
Bibliography	45

Preface

Every algorithm, however sophisticated, reduces to a finite sequence of elementary operations: additions, multiplications, comparisons. Every physical system that performs computation — whether built from vacuum tubes, transistors, optical waveguides, or any substrate yet to be conceived — implements those same operations through different physical means. What persists across all such changes of medium and representation is a layer of mathematical structure. This monograph is concerned with that layer, which remains invariant.

The question that organizes the exposition is therefore *what* is being computed — and what structural properties that object possesses independently of the coordinate system, basis, or hardware in which it happens to be expressed. This monograph began as an attempt to understand what remains invariant when formulas, coordinates, and representations are removed.

A recurring distinction will be made between an axiomatic theory and a concrete model of that theory. In a refined axiomatic treatment of Euclidean geometry, objects such as *point*, *line*, and *plane* are taken as primitive (undefined) terms, and the axioms specify only the relations they must satisfy. A Cartesian coordinate system does not redefine these primitives; rather, it provides a *model* in which points are represented by tuples in $\mathbb{R}^n$, lines by solution sets of linear equations, and incidence becomes set membership [3]. As stated in [5], this is *the assignment of meanings*. This perspective places coordinate formulas as representations of structure, and not as the structure itself. The same distinction runs through every chapter of this text.

Vector spaces are introduced as abstract sets equipped with addition and scalar multiplication. Linear transformations are defined as the maps that preserve this structure.

By introducing bilinear forms and, ultimately, inner products, vectors are allowed to interact in a way that produces scalars. From this single interaction arise norms, orthogonality, distances, and classical geometric objects such as balls, spheres, and ellipsoids.

Many constructions encountered earlier — bilinear forms, inner products, matrix multiplication — share a common feature: they are linear in each argument separately. This observation leads naturally to multilinear maps and to tensors.

Tensors are treated as abstract multilinear objects defined without coordinates. The tensor product is introduced through its universal property, explaining how multilinear problems are canonically reduced to linear ones. Index notation, arrays, and computational implementations are presented explicitly as representations.

The text is organized around four successive structural themes: linear structure, geometric and topological structure, multilinear structure, and logarithmic structure. Each theme is a

refinement of the same organizing idea: structure is revealed through invariance under transformations. The logarithmic theme, in particular, illustrates that this principle is not confined to the linear setting — the logarithm is a global isomorphism between multiplicative and additive structures, and as such it belongs to the same invariant framework as vectors and tensors.

For the reader, in order to follow the exposition and use these notes as a starting point for further research, prior familiarity is assumed with basic set theory [5][4][8], relations, functions [8], n -tuples [1][4], and basic algebra [1]. Basic knowledge of calculus [8] is also assumed. The reports [14] and [15] cover the foundational material on tensors. Other introductory material is scattered throughout the bibliography.

The topics gathered in this monograph belong to the established literature on linear algebra and abstract algebra. What motivated its writing was the perception that a single organizing idea — the preservation of structure under transformation — recurs persistently across both pure and applied science. To gather that unity into a single volume is the purpose of this work. Throughout the text, the word *invariant* is used in a broad sense: that which remains unchanged when coordinates, representations, or physical substrates are varied. The reader acquainted with algebra will recognize that the term also carries more precise technical meanings — invariant subspaces, similarity invariants, topological invariants — which should not be confused with this broader, structural usage.

An AI language model was used for editing, L^AT_EX and stylistic refinement.

This monograph presents the abstract structure that underlies every algorithm and autonomous computing system, and therefore remains invariant under any change in the state of technology. There is a more physical and hardware-oriented counterpart, centered on a 1960s computer — the Apollo Guidance Computer — which is specific to a particular moment in the history of computing and the state of the art. For the full context, see:

<https://jportillo34.github.io/ApolloGuidanceComputer/>

Chapter 1

Linear Transformations, Matrices, and Composition

$$\langle \langle K, +_g, \cdot \rangle, \langle E, +_E \rangle, \circ \rangle$$

The triple above defines a *vector space* E over the *field* K [4]. Elements of K are called *scalars*, and elements of E are called *vectors*. The operation $\circ : K \times E \rightarrow E$ is scalar multiplication, and $+_E$ is vector addition. In what follows, symbols $\alpha_1, \dots, \alpha_k$ always denote scalars in K , and symbols $x_1, \dots, x_k$ always denote vectors in E .

Informally, a vector space is a set of vectors that can be added together and scaled by elements of a field.

Example (Coordinate n -space, [6]).

Let Γ be a field. Consider the set

$$\Gamma^n = \Gamma \times \dots \times \Gamma$$

of n -tuples $(\xi^1, \dots, \xi^n)$, with $\xi^i \in \Gamma$. Define addition by

$$(\xi^1, \dots, \xi^n) + (\eta^1, \dots, \eta^n) = (\xi^1 + \eta^1, \dots, \xi^n + \eta^n)$$

and scalar multiplication by

$$\lambda(\xi^1, \dots, \xi^n) = (\lambda\xi^1, \dots, \lambda\xi^n), \quad \lambda \in \Gamma.$$

With these operations, Γ^n is a vector space over Γ , called the *n -space over Γ* . In particular, Γ itself is a vector space over Γ , where scalar multiplication coincides with field multiplication.

Given vectors $x_1, \dots, x_k \in E$ and scalars $\alpha_1, \dots, \alpha_k \in K$, an expression of the form

$$\alpha_1 \circ x_1 +_E \alpha_2 \circ x_2 +_E \dots +_E \alpha_k \circ x_k$$

is called a *linear combination*.

A subset $S \subset E$ is called a *system of generators* (or a *spanning set*) of E if every vector $x \in E$ can be written as a linear combination of elements of S [6]. That is, for every $x \in E$ there exist vectors $s_1, \dots, s_k \in S$ and scalars $\alpha_1, \dots, \alpha_k \in K$ such that

$$x = \alpha_1 \circ s_1 +_E \dots +_E \alpha_k \circ s_k.$$

A finite family of vectors $\{x_1, \dots, x_k\} \subset E$ is called *linearly independent* if the relation

$$\alpha_1 \circ x_1 +_E \dots +_E \alpha_k \circ x_k = 0$$

implies

$$\alpha_1 = \dots = \alpha_k = 0.$$

If such a nontrivial relation exists (i.e., with at least one coefficient $\alpha_i \neq 0$), the family is called *linearly dependent*.

A finite set of vectors $\{b_1, \dots, b_n\} \subset E$ is called a *basis* of the vector space E if it is both a system of generators of E and linearly independent.

Equivalently, $\{b_1, \dots, b_n\}$ is a basis of E if every vector $x \in E$ can be written *uniquely* as a linear combination

$$x = \alpha_1 \circ b_1 +_E \alpha_2 \circ b_2 +_E \dots +_E \alpha_n \circ b_n, \quad \alpha_1, \dots, \alpha_n \in K.$$

A basis therefore provides a minimal and non-redundant set of building blocks for the entire space. Once a basis is fixed, every vector is completely determined by its coefficients relative to that basis, so we can say the basis generates the space.

If the vector space E admits a finite basis, the number of vectors in any basis of E is called the *dimension* of E and is denoted by $\dim E$.

Let E and G be vector spaces over the same field K . A function $\psi : E \rightarrow G$ is called a *linear transformation* (also known as *linear map*) if it preserves linear combinations [1]:

$$\psi(\alpha_1 \circ_E x_1 + \dots + \alpha_k \circ_E x_k) = \alpha_1 \circ_G \psi(x_1) + \dots + \alpha_k \circ_G \psi(x_k).$$

Linear transformations preserve the algebraic structure of vector spaces independently of any choice of basis.

Definition (Isomorphism). Let E and F be vector spaces over the same field K . A linear transformation $\varphi : E \rightarrow F$ is called an *isomorphism* if for every $y \in F$ there exists a unique $x \in E$ such that $\varphi(x) = y$, and $\varphi(x_1) = \varphi(x_2)$ implies $x_1 = x_2$. In this case, we say that E and F are *isomorphic*.

An isomorphism preserves all linear structure: addition, scalar multiplication, linear independence, dimension, and the validity of linear relations. Hence, isomorphic vector spaces are structurally identical.

Remark (On coordinates and isomorphism [2]).

Let E be a finite-dimensional vector space over K with $\dim E = n$. Once a basis $B = \{b_1, \dots, b_n\}$ is chosen, every vector $x \in E$ is uniquely represented by its coordinate column

$$[x]_B \in K^n.$$

This correspondence defines a linear isomorphism

$$E \longrightarrow K^n.$$

Thus, from the purely algebraic point of view, every n -dimensional vector space over K is structurally indistinguishable from K^n .

However, this identification depends on the chosen basis. Different bases produce different coordinate representations. The intrinsic properties of the space are precisely those that remain invariant under such changes of reference system.

For this reason, we treat vector spaces as abstract entities, independent of any particular coordinate realization. Coordinates are tools for computation; structure is the object of study.

We define the *kernel* of ψ by

$$\ker(\psi) = \{x \in E : \psi(x) = 0\},$$

that is, the set of vectors in E that the transformation sends to the zero vector of G . We define the *image* of ψ by

$$\text{Im}(\psi) = \{\psi(x) : x \in E\} \subseteq G$$

that is, the image is the set of values in G that ψ actually attains.

If E is finite-dimensional, then

$$\dim E = \dim \ker(\psi) + \dim \text{Im}(\psi),$$

a relation known as the rank–nullity theorem.

We say that a linear transformation ψ is *injective* if $\psi(x) = \psi(y) \Rightarrow x = y$ (equivalently, $\ker(\psi) = \{0\}$), and *surjective* if for every $g \in G$ there exists $x \in E$ such that $\psi(x) = g$ (equivalently, $\text{Im}(\psi) = G$).

Let $B_E = \{e_1, \dots, e_n\}$ and $B_G = \{g_1, \dots, g_m\}$ be bases of E and G . Each image $\psi(e_i)$ can be written uniquely as $\psi(e_i) = \alpha_{1i} \circ g_1 + \dots + \alpha_{mi} \circ g_m$. The scalars α_{ji} form an $m \times n$ rectangular array

$$A = \begin{bmatrix} \alpha_{11} & \alpha_{12} & \dots & \alpha_{1n} \\ \alpha_{21} & \alpha_{22} & \dots & \alpha_{2n} \\ \vdots & \vdots & & \vdots \\ \alpha_{m1} & \alpha_{m2} & \dots & \alpha_{mn} \end{bmatrix}.$$

and its abbreviated form

$$A = (\alpha_{ij}),$$

called the *matrix of ψ relative to the chosen bases*.

Let $A = (\alpha_{ij})$ be an $m \times n$ matrix over a field K .

For each $i = 1, \dots, m$, the i th row

$$(\alpha_{i1}, \dots, \alpha_{in})$$

is called a *row vector* of A . It is naturally regarded as an element of K^n .

For each $j = 1, \dots, n$, the j th column

$$(\alpha_{1j}, \dots, \alpha_{mj})^T$$

is called a *column vector* of A . It is naturally regarded as an element of K^m .

When A represents a linear transformation $\psi : E \rightarrow G$ with respect to chosen bases, the j th column of A is precisely the coordinate vector of $\psi(e_j)$ in the basis of G .

This fact follows directly from linearity. Let $B_E = \{e_1, \dots, e_n\}$ be the chosen basis of E . Every vector $x \in E$ admits a unique representation

$$x = \alpha_1 e_1 + \dots + \alpha_n e_n.$$

Since ψ preserves linear combinations,

$$\psi(x) = \alpha_1 \psi(e_1) + \dots + \alpha_n \psi(e_n).$$

Thus the action of ψ on an arbitrary vector is completely determined by its values on the basis vectors. If the coordinates of each $\psi(e_j)$ relative to the basis of G are arranged as the j th column of a matrix A , then the coordinate vector of $\psi(x)$ is obtained by forming the same linear combination of those columns with coefficients $\alpha_1, \dots, \alpha_n$. In coordinates,

$$[\psi(x)]_{B_G} = A[x]_{B_E}.$$

Thus a linear transformation is completely determined by the images of the basis vectors, and the matrix records precisely these images in coordinate form.

Matrices are concrete representations of linear transformations once bases have been chosen.

Linear transformations may also be applied successively. Understanding how their coordinate representations combine leads naturally to matrix multiplication.

Matrix multiplication is the coordinate expression of *composition of linear maps*.

Derivation (Matrix multiplication from composition).

Let

$$S : K^p \longrightarrow K^n, \quad T : K^n \longrightarrow K^m$$

be linear transformations with matrices (relative to chosen bases)

$$B \in M_{n \times p}(K), \quad A \in M_{m \times n}(K).$$

For a vector $x \in K^p$, matrix–vector multiplication represents application of the corresponding linear map:

$$S(x) = Bx, \quad T(y) = Ay.$$

The composition $T \circ S : K^p \rightarrow K^m$ therefore satisfies

$$(T \circ S)(x) = T(S(x)) = A(Bx).$$

Since a matrix represents a linear transformation uniquely in coordinates, there must exist a matrix C such that

$$(T \circ S)(x) = Cx \quad \text{for all } x \in K^p.$$

By definition, this matrix is the product $C = AB$, and it is determined by the requirement

$$(AB)x = A(Bx).$$

Writing this relation in coordinates shows that the entries of $C = AB$ are necessarily given by

$$(AB)_{ij} = \sum_{k=1}^n A_{ik} B_{kj}, \quad 1 \leq i \leq m, \quad 1 \leq j \leq p.$$

Thus the familiar rule for matrix multiplication is the coordinate expression of composition of linear maps.

In many practical cases, a vector does not merely represent an abstract element of a vector space, but the *state* of a system expressed relative to a chosen basis. We will refer to such a choice of basis as a *reference system*. This framing adds no new algebraic content, but it motivates a natural question: what happens to the coordinate representation of a state when the reference system changes?

Let E be a finite-dimensional vector space and let

$$B = \{e_1, \dots, e_n\}$$

be a basis of E . Any vector $x \in E$ can be written uniquely as

$$x = \alpha_1 \circ e_1 +_E \dots +_E \alpha_n \circ e_n.$$

The scalars $\alpha_1, \dots, \alpha_n$ are the *coordinates* of the state x relative to the reference system B . Writing these coordinates as a column vector,

$$[x]_B = \begin{bmatrix} \alpha_1 \\ \vdots \\ \alpha_n \end{bmatrix},$$

is a notational convenience: it encodes the action of linear combinations relative to the chosen basis.

Suppose now that a second reference system

$$B' = \{e'_1, \dots, e'_n\}$$

is chosen for the same space E . The vector x is the same abstract object, but its coordinates relative to B' form a different column vector $[x]_{B'}$.

The passage from $[x]_B$ to $[x]_{B'}$ is not arbitrary. It is determined by a linear transformation

$$T : K^n \longrightarrow K^n,$$

whose matrix depends only on the relation between the two bases. Thus,

$$[x]_{B'} = A[x]_B,$$

where A is the matrix of the change of reference system. This is a direct instance of the matrix–vector product $[\psi(x)]_{B_G} = A[x]_{B_E}$ derived above: here the “transformation” is precisely the relabelling of coordinates induced by passing from B to B' .

In coordinates, each row of the matrix represents the coordinate expression of a linear functional on K^n (we will formalize this identification when discussing dual spaces), and matrix–vector multiplication consists precisely in applying each of these functionals to the coordinate vector. Concretely, if A is an $m \times n$ matrix and x is a column vector in K^n , then the i th component of Ax is the value of the functional given by the i th row of A applied to x ; this is defined only when x has n components, i.e., only when the number of columns of A equals the number of rows of x .

The requirement that the number of columns of A equal the number of components of $[x]_B$ is not a convention; it expresses the fact that the codomain of one linear map must match the domain of the next.

In this sense, column vectors represent states, matrices represent transformations between reference systems, and matrix multiplication represents successive changes of coordinates. The familiar computational rules are therefore consequences of the abstract structure introduced earlier.

Linear transformations therefore form the structural backbone of many quantitative models. Even in systems where nonlinear effects dominate, computations are typically organized around repeated applications and compositions of linear maps. The linear framework therefore provides a universal computational language, even when the phenomena being modeled are not themselves linear.

The derivation of matrix multiplication revealed that a row vector acts on a column vector to produce a scalar: each row of a matrix defines a rule that assigns a number to any element of K^n , and this rule is linear. This suggests that row vectors and column vectors are fundamentally different kinds of objects. Making this distinction precise requires introducing the *dual space*.

The set of all linear transformations from E into the base field K is denoted by

$$E^* = L(E, K)$$

and is called the *dual space* of E . An element $x^* \in E^*$ is called a *linear functional*. Thus,

$$x^* : E \rightarrow K$$

satisfies

$$x^*(\alpha_1 \circ_E x_1 +_E \cdots +_E \alpha_k \circ_E x_k) = \alpha_1 x^*(x_1) + \cdots + \alpha_k x^*(x_k).$$

Linear functionals assign scalar values to vectors while preserving linear structure.

If E is finite-dimensional with basis $B_E = \{e_1, \dots, e_n\}$, then every $x^* \in E^*$ is uniquely determined by

$$x^*(e_1), \dots, x^*(e_n).$$

These scalars form a row vector

$$[x^*(e_1) \ \dots \ x^*(e_n)].$$

In contrast, vectors in E are represented as column vectors. These two objects live in different spaces: E and E^* .

Given a matrix

$$A = (\alpha_{ij}) \in M_{m \times n}(K),$$

its *transpose* is the matrix

$$A^T = (\alpha_{ji}) \in M_{n \times m}(K),$$

obtained by interchanging rows and columns.

Thus the i th row of A becomes the i th column of A^T , and the j th column of A becomes the j th row of A^T .

The transpose operation exchanges the roles of rows and columns. In particular, if a matrix represents a linear map $\psi : K^n \rightarrow K^m$, then its transpose has the reversed size and may be viewed, purely at the level of arrays, as interchanging the domain and codomain indices.

Column vectors represent elements of a vector space E , whereas row vectors represent linear functionals, elements of the dual space E^* . These are algebraically different objects, even though they may have the same number of components.

In many situations, however, a vector is used to define a linear functional. This requires additional structure (Chapter II).

A column vector represents an element of K^n , a row vector represents an element of $(K^n)^*$, and an $m \times n$ matrix represents a linear map

$$K^n \xrightarrow{\psi} K^m.$$

A product such as

$$[x^*] A$$

is defined because it corresponds to the composition

$$K^n \xrightarrow{\psi} K^m \xrightarrow{x^*} K$$

The dimensions match precisely because the codomain of the first map equals the domain of the second.

By contrast, a product of two row vectors would attempt to compose

$$K^n \rightarrow K \quad \text{with} \quad K^n \rightarrow K,$$

which is meaningless: the output of the first map does not lie in the domain of the second. The product is therefore undefined by necessity.

Matrix dimensions encode compatibility of domains and codomains. One may multiply matrices if and only if the corresponding linear maps can be composed.

Concrete numerical example.

Let

$$x^*(x, y) = 3x + 2y$$

be a linear functional on K^2 . Relative to the canonical basis, it is represented by the row vector

$$[3 \ 2].$$

Let

$$A = \begin{bmatrix} 2 & 3 \\ 5 & 4 \end{bmatrix}$$

be the matrix of a linear transformation $\psi : K^2 \rightarrow K^2$.

The product

$$[3 \ 2]A = [3 \ 2] \begin{bmatrix} 2 & 3 \\ 5 & 4 \end{bmatrix} = [19 \ 17]$$

is defined because it represents the composition

$$K^2 \xrightarrow{\psi} K^2 \xrightarrow{x^*} K.$$

The result is again a row vector, corresponding to the linear functional

$$(x, y) \mapsto 19x + 17y.$$

By contrast, the product

$$[3 \ 2][a \ b]$$

is undefined because it would attempt to compose two maps of the form

$$K^2 \rightarrow K,$$

which is impossible: the output of the first map is a scalar, not a vector in K^2 .

Conceptual summary. The structures introduced in this chapter—vector spaces, linear transformations, bases, coordinate systems, and dual spaces—are all facets of the same underlying linear framework. Matrices encode linear transformations relative to a chosen basis, column vectors encode states relative to a reference system, and systems of linear equations are simply coordinate expressions of these transformations asking whether a vector lies in the image of a map. Algorithms such as Gaussian elimination, while essential for computation, do not create new structures: they explore the coordinate-level consequences of the abstract linear relationships already present. Together, these concepts illustrate how abstract algebraic definitions give rise to the familiar computational and geometric tools used in science and engineering.

Notes and Remarks

Note (Why linearity?, [7]).

An elementary reason for the foundational nature of linear algebra within mathematics is that, under suitable conditions, functions can be locally approximated by linear functions. This fact, together with the tendency of natural systems to reside near the minimum of their potential energy, explains the prevalence of linearity in scientific phenomena.

For now, we will say informally that a situation exhibits linearity if we can identify a numerical output and a numerical input such that the former is proportional to the latter.

Note: Direct Sums and Decomposition of Vector Spaces [2]

Given two vector spaces U and V over the same field K , their *direct sum* is the vector space

$$U \oplus V = \{(x, y) \mid x \in U, y \in V\},$$

with addition and scalar multiplication defined componentwise.

The subspaces

$$\{(x, 0) \mid x \in U\} \quad \text{and} \quad \{(0, y) \mid y \in V\}$$

may be identified with U and V , respectively. Every element of $U \oplus V$ admits a unique decomposition

$$(x, y) = (x, 0) + (0, y),$$

so that $U \oplus V$ is obtained by combining two independent components.

More generally, a vector space W is said to be the direct sum of subspaces U and V if every $w \in W$ can be written uniquely as

$$w = u + v, \quad u \in U, v \in V.$$

In this case one writes $W = U \oplus V$.

The notion of direct sum expresses a structural principle: a space may be decomposed into independent parts whose contributions combine additively without interaction.

This construction should be distinguished from algebraic extensions. For example, the complex numbers $\mathbb{C}$ form a two-dimensional vector space over $\mathbb{R}$ and may be identified, as vector spaces, with $\mathbb{R} \oplus \mathbb{R}$ via

$$a + ib \longleftrightarrow (a, b).$$

However, this identification does not preserve multiplication. The product of complex numbers couples the two components, introducing an interaction that is not present in the direct sum.

Thus, while some extensions can be viewed as direct sums at the level of vector spaces, additional algebraic structure may introduce relations between components that go beyond linear combination.

The direct sum provides the fundamental way of assembling independent linear components. Later constructions, such as the tensor product, will introduce a different kind of combination in which components interact multiplicatively rather than additively.

Note (Systems of linear equations).

Operationally, a system of linear equations of the form

$$\begin{cases} a_{11}x_1 + \cdots + a_{1n}x_n = b_1 \\ \vdots \\ a_{m1}x_1 + \cdots + a_{mn}x_n = b_m. \end{cases}$$

is treated as a collection of equations to be solved for the unknowns $x_1, \dots, x_n$.

From the abstract point of view, this system is nothing more than the coordinate expression of a linear transformation.

Indeed, let

$$\psi : K^n \longrightarrow K^m$$

be the linear transformation whose matrix relative to the canonical bases is

$$A = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & & \vdots \\ a_{m1} & \cdots & a_{mn} \end{bmatrix}.$$

Writing

$$x = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} \quad \text{and} \quad b = \begin{bmatrix} b_1 \\ \vdots \\ b_m \end{bmatrix},$$

the system above is equivalently expressed as the single equation

$$\psi(x) = b, \quad \text{or, in coordinates,} \quad Ax = b.$$

So, a system of linear equations specifies a vector $b \in K^m$ and asks whether it lies in the image of a linear transformation $\psi : K^n \rightarrow K^m$. Questions of existence, uniqueness, or multiplicity of solutions correspond to geometric properties of this map, such as injectivity and surjectivity, and are independent of any particular algorithm used to compute them.

Algorithms such as Gaussian elimination are methods for finding a vector $x \in K^n$ that satisfies $Ax = b$ relative to a chosen reference system (basis). From the abstract perspective, they are simply procedures for exploring the image and kernel of a linear transformation in coordinates. That is, Gaussian elimination exploits the structure of the linear map encoded by the matrix A to determine whether a solution exists, whether it is unique, and how to express all possible solutions, all while working entirely in the chosen coordinate system.

Note (Matrix as a function, [1]).

Let $S \neq \emptyset$ be a set and let $m, n \geq 1$. An $m \times n$ matrix with entries in S is a function

$$M : \{1, \dots, m\} \times \{1, \dots, n\} \longrightarrow S.$$

We write $a_{ij} = M(i, j)$ and abbreviate M by $M = (a_{ij})$ (or $M = (a_{ij})_{m \times n}$) when no confusion can arise.

Specialization. When $S = K$ is a field, the set of all $m \times n$ matrices over K is denoted by $M_{m \times n}(K)$.

Note (Generalized matrix product, [11]).

There is a useful abstraction of matrix multiplication that makes explicit which algebraic properties are really being used. If A is an $m \times n$ matrix and B is an $n \times s$ matrix, and if $\circ$ and $\bullet$ are binary operations such that $\circ$ is associative, one may define the *generalized matrix product*

$$A \overset{\circ}{\underset{\bullet}{\bullet}} B$$

to be the $m \times s$ matrix C whose entries are

$$C_{ij} = (A_{i1} \bullet B_{1j}) \circ (A_{i2} \bullet B_{2j}) \circ \dots \circ (A_{in} \bullet B_{nj}), \quad 1 \leq i \leq m, \ 1 \leq j \leq s.$$

The ordinary matrix product is obtained when $\circ$ is $+$ and $\bullet$ is $\cdot$.

Example (Polynomials as a free linear construction, [10]).

Let K be a field. Consider the set

$$K[x] = \left\{ \sum_{k=0}^n a_k x^k \mid n \in \mathbb{N}, \ a_k \in K \right\},$$

whose elements are called *polynomials* in the formal symbol x with coefficients in K .

The symbol x is not assigned a numerical value; it serves only as a generator of formal expressions. Addition and scalar multiplication are defined coefficientwise:

$$\left(\sum_{k=0}^n a_k x^k\right) +_{K[x]} \left(\sum_{k=0}^m b_k x^k\right) = \sum_{k=0}^{\max(n,m)} (a_k +_K b_k) x^k,$$

$$\alpha \circ \left(\sum_{k=0}^n a_k x^k\right) = \sum_{k=0}^n (\alpha \cdot a_k) x^k.$$

With these operations, $K[x]$ becomes a vector space over K .

The family

$$\{1, x, x^2, x^3, \dots\}$$

is linearly independent and generates $K[x]$. Hence it forms a basis of $K[x]$.

Every polynomial can therefore be written uniquely as a linear combination

$$u(x) = a_0 \circ 1 +_{K[x]} a_1 \circ x +_{K[x]} \dots +_{K[x]} a_n \circ x^n, \quad a_k \in K.$$

The vector space $K[x]$ is infinite dimensional, since its basis contains infinitely many elements.

Conceptually, $K[x]$ is the vector space freely generated by the formal powers

$$\{x^k\}_{k \geq 0}.$$

No relations exist among these generators other than those required by the axioms of a vector space.

Polynomial multiplication introduces an additional operation on $K[x]$,

$$x^i \cdot x^j = x^{i+j},$$

which is compatible with the linear structure but is not required for the definition of the vector space itself.

Thus, polynomials provide a canonical example of a linear structure generated by formal symbols. The linear framework appears independently of geometry, coordinates, or numerical interpretation. Multiplication enriches this structure, but linearity is primary.

Chapter 2

Geometry from Linear Structure

In Chapter I, vector spaces were introduced as purely algebraic objects. A vector was an abstract element of a set equipped with addition and scalar multiplication, and linear transformations were functions preserving this structure. No notion of length, angle, distance, or orthogonality was assumed or required.

Algebra alone allows us to decide whether vectors are independent, whether a set spans a space, or whether a linear transformation is injective or surjective. However, it does not allow us to ask geometric questions. In many situations, it is not enough to know *which* vectors are possible; we also need to know how *large* they are, how *close* they are to one another, or how *aligned* they are. These notions are expressed through length, distance, and angle. Without additional structure, the language of vector spaces does not distinguish between “larger” and “smaller,” nor does it allow one to speak of perpendicularity or of the “best approximation” of one vector by others (as in projections and least squares). There is no intrinsic way, within a bare vector space, to compare the size of two vectors or to determine whether they point in perpendicular directions.

Geometry begins only when additional structure is imposed.

It is useful now to recall how geometric space is commonly represented in coordinates. Given sets $A_1, \dots, A_n$, their *Cartesian product* (also known as *Product Set*) is

$$A_1 \times \cdots \times A_n = \{(x_1, \dots, x_n) \mid x_i \in A_i\}.$$

In particular, the coordinate space $\mathbb{R}^n$ is the Cartesian product of n copies of $\mathbb{R}$. This construction by itself is not yet geometry: it provides a convenient *model* in which “points” are represented by tuples of real numbers. A geometry emerges only after we specify an additional rule that allows us to measure and compare such tuples.

A central theme of this document is that geometry is not a replacement for linear structure, but an enrichment of it. The vector spaces introduced earlier remain unchanged; what changes is that they are now equipped with a rule that allows vectors to interact in a way that produces scalar quantities.

Informally, this rule takes two vectors and returns a scalar. From this single operation, notions such as length, angle, and distance emerge naturally.

To make this interaction precise, we introduce the notion of a bilinear map.

Let E and G be vector spaces over the same field K . A mapping

$$B : E \times G \longrightarrow K$$

is called *bilinear* if it is linear in each argument separately. That is, for fixed $y \in G$, the map

$$x \longmapsto B(x, y)$$

is a linear functional on E , and for fixed $x \in E$, the map

$$y \longmapsto B(x, y)$$

is a linear functional on G .

Explicitly, for all $x_1, x_2 \in E$, $y_1, y_2 \in G$, and all scalars $\alpha \in K$,

$$\begin{aligned} B(x_1 + x_2, y) &= B(x_1, y) + B(x_2, y), & B(\alpha x, y) &= \alpha B(x, y), \\ B(x, y_1 + y_2) &= B(x, y_1) + B(x, y_2), & B(x, \alpha y) &= \alpha B(x, y). \end{aligned}$$

When $E = G$, such a mapping is called a *bilinear form* on E .

At this level, no geometric interpretation is assumed. A bilinear form is merely a rule that assigns a scalar to an ordered pair of vectors, subject to linearity in each argument. It need not be symmetric, positive, or related to any notion of length or angle.

Nevertheless, bilinear forms play a fundamental structural role. They provide a systematic way to associate vectors with linear functionals and to construct scalar quantities from pairs of vectors.

Only after additional conditions are imposed on a bilinear form do familiar geometric notions emerge. These special cases will be introduced shortly.

Up to this point, the scalar field K has been arbitrary. Pure linear structure does not require the ability to compare scalars. Geometry, however, depends on positivity: expressions such as $\langle x, x \rangle > 0$ and inequalities of the form $r > 0$ presuppose an order relation. For this reason, we now restrict attention to the real number system

$$\langle \mathbb{R}, +, \cdot, \leq \rangle,$$

viewed as an ordered field. It is precisely the order on $\mathbb{R}$ that allows lengths, distances, and orthogonality to acquire geometric meaning.

We now specialize the previously introduced bilinear forms.

Let E be a vector space over $\mathbb{R}$. A *bilinear form* on E is a function

$$B : E \times E \longrightarrow \mathbb{R}$$

that is linear in each argument separately. That is, for all $x, x', y, y' \in E$ and all $\lambda \in \mathbb{R}$,

$$B(x + x', y) = B(x, y) + B(x', y), \quad B(\lambda x, y) = \lambda B(x, y),$$

and similarly in the second argument.

Bilinear forms arise naturally from linear transformations. If

$$\psi : E \longrightarrow E^*$$

is a linear mapping, then the formula

$$B(x, y) = \psi(x)(y)$$

defines a bilinear form on E . Conversely, every bilinear form determines such a mapping. Therefore, bilinear forms encode a controlled way in which vectors may interact to produce scalars.

At this level, no geometry is present. A bilinear form may be degenerate, asymmetric, or indefinite. It allows vectors to interact, but it does not yet measure anything.

Geometry appears when the bilinear form satisfies additional constraints.

An *inner product* on a vector space E over $\mathbb{R}$ is a bilinear form

$$\langle \cdot, \cdot \rangle : E \times E \longrightarrow \mathbb{R}$$

that is symmetric, positive, and non-degenerate. These properties ensure that

$$\langle x, x \rangle > 0 \quad \text{for all } x \neq 0.$$

Once such a rule is fixed, vectors acquire measurable properties. Two vectors may be declared orthogonal. A vector may be assigned a length. The distance between two points may be defined in terms of the vectors connecting them. None of these concepts exist prior to the introduction of this additional structure.

As in Chapter I, coordinates play no fundamental role. Geometric quantities must be independent of any particular reference system. A formula that changes when the basis is changed describes a representation, not a geometric object.

Only after a basis is chosen do familiar coordinate expressions appear. Sums of squares, dot products, and quadratic equations arise as coordinate-level manifestations of the underlying structure, not as definitions of it.

Coordinate expression of the inner product.

Let $(E, \langle \cdot, \cdot \rangle)$ be a finite-dimensional inner product space and let $\{e_1, \dots, e_n\}$ be an orthonormal basis. Every vector $x \in E$ can be written uniquely as

$$x = \sum_{i=1}^n x_i e_i, \quad y = \sum_{i=1}^n y_i e_i.$$

Using bilinearity and the orthonormality condition $\langle e_i, e_j \rangle = \delta_{ij}$, we obtain

$$\langle x, y \rangle = \sum_{i=1}^n x_i y_i.$$

Thus, in an orthonormal basis the abstract inner product is represented by the familiar *dot product* of coordinate vectors in $\mathbb{R}^n$.

This observation explains the role of matrix multiplication. If a linear transformation $\psi : E \rightarrow G$ is represented by a matrix $A = (a_{ij})$ and a vector x has coordinate vector $x = (x_1, \dots, x_n)^T$, then the i -th component of the product Ax is

$$(Ax)_i = \sum_{j=1}^n a_{ij} x_j.$$

Each component is therefore the dot product of the i -th row of A with the vector x . Matrix multiplication can thus be understood as the systematic evaluation of inner products between rows of one matrix and columns of another.

In this sense, the familiar algebraic rule for multiplying matrices is not an arbitrary computational convention: it admits a natural geometric interpretation as the coordinate expression of interactions measured by an inner product.

This point of view explains why objects such as circles, spheres, and ellipsoids can be described by algebraic equations, yet retain a geometric meaning that is invariant under changes of coordinates. The equations depend on the chosen basis; the geometry does not.

The goal is therefore to show how geometry emerges from linear structure once a suitable interaction between vectors is specified.

Although geometry has begun to emerge through bilinear interaction, the underlying algebraic mechanism remains the linear transformation. To understand how geometric structure interacts with linear maps, we examine a construction naturally associated with every linear transformation.

The central object of this work is the linear transformation. Before introducing geometric notions such as length, angle, and distance, we make explicit a fundamental construction associated with every linear transformation: its transpose.

Let E and G be vector spaces over the same field K , and let

$$\psi : E \longrightarrow G$$

be a linear transformation. Recall that the dual spaces E^* and G^* consist of all linear functionals on E and G , respectively.

For each $g^* \in G^*$, the composition

$$g^* \circ \psi : E \longrightarrow K$$

is a linear functional on E . This defines a mapping

$$\psi^* : G^* \longrightarrow E^*, \quad \psi^*(g^*) = g^* \circ \psi.$$

The mapping ψ^* is linear and is called the *transpose* (or dual transformation) of ψ . The direction of the arrow is reversed:

$$E \xrightarrow{\psi} G \quad \Longrightarrow \quad G^* \xrightarrow{\psi^*} E^*.$$

This construction is intrinsic: it depends only on the linear transformation ψ and does not require any choice of basis, coordinates, or inner product.

When bases are chosen for E and G , and the corresponding dual bases are chosen for E^* and G^* , the linear transformation ψ is represented by a matrix A , and the dual transformation ψ^* is represented by the transposed matrix A^T . Thus, the familiar matrix transpose is the coordinate expression of the intrinsic dual transformation ψ^* .

The inner product does more than assign lengths. It establishes a canonical identification between the vector space and its dual.

For each vector $x \in E$, define the mapping

$$\Phi(x) : E \longrightarrow \mathbb{R}, \quad \Phi(x)(y) = \langle x, y \rangle.$$

By bilinearity, $\Phi(x)$ is a linear functional on E . Thus, we obtain a mapping

$$\Phi : E \longrightarrow E^*.$$

If the inner product is non-degenerate, this mapping is injective. In finite dimensions, since $\dim E = \dim E^*$, it is therefore an isomorphism.

Hence, once an inner product is fixed, every vector determines a unique linear functional, and every linear functional arises from a unique vector. The distinction between E and E^* disappears at the structural level.

It is precisely this identification that allows the transpose of a linear transformation to be regarded as an operator on E itself. When geometry is introduced, duality becomes internal.

Once an inner product is fixed, vectors acquire length. The norm of a vector $x \in E$ is defined by

$$\|x\| = \sqrt{\langle x, x \rangle}.$$

So, the vector space E introduced in Chapter I is now equipped with additional geometric structure:

$$\left\langle \langle \mathbb{R}, +, \cdot \rangle, \langle E, + \rangle, \circ, \langle \cdot, \cdot \rangle \right\rangle.$$

The inner product is not required for linearity itself, but it allows the introduction of geometric notions such as length, angle, orthogonality, and adjoint transformations.

Algebraic expansion of the norm.

For any vectors $x, y \in E$, the square of the norm of their sum satisfies

$$\|x + y\|^2 = \langle x + y, x + y \rangle.$$

By bilinearity and symmetry of the inner product,

$$\langle x + y, x + y \rangle = \langle x, x \rangle + \langle x, y \rangle + \langle y, x \rangle + \langle y, y \rangle = \|x\|^2 + 2\langle x, y \rangle + \|y\|^2.$$

Thus,

$$\|x + y\|^2 = \|x\|^2 + \|y\|^2 + 2\langle x, y \rangle.$$

This identity holds in every inner product space. It expresses how the interaction term $\langle x, y \rangle$ measures the deviation from pure additivity of squared lengths.

The inner product also satisfies the inequality

$$|\langle x, y \rangle| \leq \|x\| \|y\|,$$

known as the *Cauchy–Schwarz inequality*. As a consequence,

$$\|x + y\|^2 \leq \|x\|^2 + \|y\|^2 + 2\|x\| \|y\| = (\|x\| + \|y\|)^2.$$

Taking square roots yields the *triangle inequality*

$$\|x + y\| \leq \|x\| + \|y\|.$$

The *Pythagorean identity* arises as the special case $\langle x, y \rangle = 0$, in which the interaction term vanishes and

$$\|x + y\|^2 = \|x\|^2 + \|y\|^2.$$

Orthogonality also becomes meaningful. Two vectors $x, y \in E$ are said to be orthogonal if

$$\langle x, y \rangle = 0.$$

The order of the vectors is irrelevant. Symmetry of the inner product implies

$$\langle x, y \rangle = \langle y, x \rangle,$$

so orthogonality is a mutual relation.

The zero vector is orthogonal to every vector in E . Indeed, for any $v \in E$,

$$\langle 0, v \rangle = \langle 0 \cdot v, v \rangle = 0\langle v, v \rangle = 0.$$

Moreover, the zero vector is the only vector with this property.

Thus, orthogonality is precisely the condition under which the interaction term vanishes. In that case,

$$\|x + y\|^2 = \|x\|^2 + \|y\|^2.$$

Squared lengths become additive exactly when the vectors do not interact.

Interlude: Invariant Directions

Up to this point, linear transformations have been treated algebraically: a map $\psi : E \rightarrow E$ sends vectors to vectors while preserving linear structure. No geometric meaning has yet been attached to this action.

Once an inner product is introduced, however, the action of a linear transformation can be examined geometrically. Given a vector $x \in E$, the image $\psi(x)$ may differ from x in two independent ways: its length may change, and its direction may change.

For a generic vector, both effects occur simultaneously. Most vectors are stretched and rotated.

A remarkable phenomenon occurs when there exists a nonzero vector $x \in E$ such that

$$\psi(x) = \lambda x$$

for some scalar λ . In this case, the transformation acts on x by pure scaling. The direction of x is preserved.

Such a vector is called an *eigenvector* of ψ , and the scalar λ is its corresponding *eigenvalue*. Eigenvectors identify directions in the space that are geometrically invariant under the transformation.

This invariance is an intrinsic property of the linear transformation itself. Changing the basis may alter the representation of ψ , but it cannot destroy or create eigenvectors.

Under the identification $E \cong E^*$ induced by the inner product, the transpose ψ^* corresponds to a linear operator on E , called the *adjoint* of ψ .

When a linear transformation arises from a symmetric bilinear form (or equivalently, from a self-adjoint operator), its eigenvectors corresponding to distinct eigenvalues are orthogonal. These directions define a preferred coordinate system dictated by the geometry, not by arbitrary choice.

It is in this sense that eigenvectors serve as the principal axes of the ellipsoids associated with quadratic forms. Diagonalization is the act of aligning coordinates with the invariant directions already present in the structure.

Distance and the emergence of geometry.

Once an inner product is available on a vector space E , it determines a norm, and from the norm we obtain a notion of distance between vectors,

$$d(x, y) = \|x - y\|.$$

At this point, no new structure is introduced: distance is merely a reformulation of algebraic data already present.

This distance allows us to describe proximity. For a vector $x \in E$ and a real number $r > 0$, we consider the set

$$B(x, r) = \{ y \in E \mid d(x, y) < r \}.$$

Such a set is called an *open ball* centered at x with radius r . Geometrically, it consists of all vectors that lie closer to x than the prescribed threshold.

The strict inequality is essential. It guarantees that every point inside the ball admits further nearby points, so that no boundary is included. This “breathing room” is what ultimately allows us to speak of continuity and deformation.

The collection of all open balls determines which subsets of E should be called open: a set is open if, around each of its points, it contains some open ball. In this way, distance generates a notion of openness without any additional axioms.

This notion of openness can be captured by a small set of axioms that preserve its essential properties while allowing the concept to be extended far beyond the metric setting. The resulting structure is what is known as a *topology* on E .

A *topological space* is an ordered pair (X, τ) , where X is a set and τ is a collection of subsets of X . For τ to be a *topology*, it must satisfy the following axioms:

1. $\emptyset \in \tau$ and $X \in \tau$.
2. The union of any collection of sets in τ is also in τ .
3. The intersection of any *finite* number of sets in τ is also in τ .

The elements of τ are called *open sets*.

Thus, topology does not appear here as an independent theory, but as a structural consequence of the metric induced by the inner product. Geometry gives rise to distance, distance gives rise to neighborhoods, and neighborhoods give rise to topological structure.

Geometric interpretation.

In $\mathbb{R}^n$ equipped with its standard inner product, open balls correspond to the familiar spheres of Euclidean geometry. More general norms deform these balls into ellipsoidal shapes, reflecting anisotropic scaling along different directions of the space. In all cases, each point

$$x = (x_1, \dots, x_n)$$

is a vector of the ambient vector space, and the geometric objects considered are subsets of that space defined by distance relations among its vectors.

Once objects have been represented as vectors in this way, the geometric structures introduced above—norms, distances, balls, and ellipsoids—become tools for organizing and comparing them.

Their relative positions, measured by a chosen metric, encode similarity. This metric need not be Euclidean: in practice it may be designed to reflect domain-specific invariances or even learned from data. Neighborhoods of meaning are then described by open balls, and learning can be interpreted as the discovery of a geometric and topological organization underlying the data.

Once a norm has been introduced, certain geometric sets arise as direct consequences of the structure and require no additional assumptions.

Let $(E, \langle \cdot, \cdot \rangle)$ be an inner product space and let $\|x\| = \sqrt{\langle x, x \rangle}$ be the associated norm. For a fixed vector $x_0 \in E$ and a scalar $r > 0$, the *closed ball* of radius r centered at x_0 is defined by

$$B_r(x_0) = \{x \in E \mid \|x - x_0\| \leq r\}.$$

The corresponding *sphere* is the boundary of this set,

$$S_r(x_0) = \{x \in E \mid \|x - x_0\| = r\}.$$

Remark (Coordinate description in $\mathbb{R}^n$): When $E = \mathbb{R}^n$ is equipped with its standard inner product and we use the standard coordinates $x = (x_1, \dots, x_n)$, the same intrinsic definitions take the familiar “sum of squares” form. For a ball centered at the origin,

$$B_r(0) = \{x \in \mathbb{R}^n \mid x_1^2 + \dots + x_n^2 \leq r^2\},$$

and its boundary sphere is

$$S_r(0) = \{x \in \mathbb{R}^n \mid x_1^2 + \dots + x_n^2 = r^2\}.$$

This is simply the coordinate expression of $\|x\| = \sqrt{x_1^2 + \dots + x_n^2}$.

These definitions are intrinsic and do not depend on any choice of coordinates or basis.

Remark (Symmetry): In the Euclidean case $E = \mathbb{R}^n$ with its standard inner product, the ball is invariant under orthogonal transformations. If $Q \in O(n)$, then $\|Qx\| = \|x\|$, and hence

$$Q(B_r(0)) = B_r(0).$$

More generally, any isometry of an inner product space carries balls to balls.

Remark (Slicing in $\mathbb{R}^n$): In $\mathbb{R}^n$, fixing the last coordinate $x_n = t$ shows that the cross-section of a radius- r ball by the hyperplane $x_n = t$ is an $(n-1)$ -dimensional ball of radius $\sqrt{r^2 - t^2}$. This observation is the geometric content behind the common “integrate by slices” approach to n -dimensional volume.

Remark (Volume of Euclidean balls): In $\mathbb{R}^n$, the n -dimensional volume of a Euclidean ball scales as a constant times r^n :

$$\text{Vol}_n(B_r(0)) = \omega_n r^n,$$

where $\omega_n = \text{Vol}_n(B_1(0))$ is the volume of the unit ball. A closed form is

$$\omega_n = \frac{\pi^{n/2}}{\Gamma(\frac{n}{2} + 1)}.$$

In particular, $\omega_1 = 2$, $\omega_2 = \pi$, and $\omega_3 = \frac{4\pi}{3}$.

Remark (A high-dimensional effect): Although $B_1(0) \subset \mathbb{R}^n$ always contains many points, its n -dimensional volume satisfies $\omega_n \rightarrow 0$ as $n \rightarrow \infty$. This illustrates a basic phenomenon of high-dimensional geometry: sets that look large in low dimensions can occupy a vanishing fraction of ambient volume as the dimension increases.

More generally, let $B : E \times E \rightarrow \mathbb{R}$ be a symmetric positive-definite bilinear form. The set

$$\mathcal{E} = \{x \in E \mid B(x - x_0, x - x_0) \leq r^2\}$$

is called an *ellipsoid*. Thus, an ellipsoid is the unit ball associated with a bilinear form, or equivalently, with a modified inner product on E .

When the bilinear form B is symmetric and positive-definite, there exists a unique self-adjoint linear operator $A : E \rightarrow E$ such that

$$B(x, y) = \langle Ax, y \rangle$$

for all $x, y \in E$.

The operator A encodes the geometry of the ellipsoid. Since A is self-adjoint on a finite-dimensional inner product space, it admits an orthonormal basis of eigenvectors. If $\{e_1, \dots, e_n\}$ is such a basis and $Ae_i = \lambda_i e_i$ with $\lambda_i > 0$, then the defining inequality

$$B(x - x_0, x - x_0) \leq r^2$$

becomes

$$\sum_{i=1}^n \lambda_i \xi_i^2 \leq r^2,$$

where $\xi_i = \langle x - x_0, e_i \rangle$ are the coordinates in this eigenbasis.

Thus, the principal axes of the ellipsoid are precisely the eigenvectors of A , and the eigenvalues determine the scaling along those directions. Diagonalization does not create the axes; it merely reveals the invariant directions already present in the structure.

Accordingly, when a basis is chosen in which the bilinear form is diagonal, this invariant definition reduces to a familiar coordinate expression. In $\mathbb{R}^n$, one obtains

$$\mathcal{E} = \left\{ (x_1, \dots, x_n) \in \mathbb{R}^n \mid \sum_{i=1}^n \frac{(x_i - a_i)^2}{h_i^2} \leq r^2 \right\}.$$

Written componentwise, this inequality reads

$$\left(\frac{x_1 - a_1}{h_1} \right)^2 + \left(\frac{x_2 - a_2}{h_2} \right)^2 + \dots + \left(\frac{x_n - a_n}{h_n} \right)^2 \leq r^2.$$

The equation depends on the chosen coordinates; the ellipsoid itself does not.

Remark (Degenerate cases). The geometric nature of the set defined above depends on the coefficients $h_1, \dots, h_n$. If $h_1 = \dots = h_n$, the ellipsoid reduces to an n -dimensional sphere. If

one or more of the parameters h_i vanish, the defining quadratic form becomes degenerate and the ellipsoid collapses into a lower-dimensional object: a disk, a line segment, or, in the extreme case, a single point.

From the structural point of view, these degenerations correspond to a loss of rank in the associated quadratic or bilinear form. Thus, changes in the algebraic structure translate directly into changes in geometric dimension.

Remark. Throughout this discussion, the ambient space $E = \mathbb{R}^n$ is a vector space in the sense of Chapter I. Each element

$$x = (x_1, \dots, x_n)$$

is therefore a vector of E .

However, the sets defined by these expressions are *subsets* of E selected by quadratic constraints. They are not vector spaces themselves, since they are not closed under addition or scalar multiplication.

Conceptual summary. Vector spaces provide the stage. Linear transformations describe allowable changes of state. Geometry enters only when vectors are permitted to interact through a structured bilinear rule that produces scalars. From this interaction, lengths, angles, distances, and geometric shapes emerge. The transition from algebra to geometry is therefore not a leap, but a controlled enrichment of structure. The book [3] is a wonderful reference on the topology of metric spaces.

Notes and Remarks

Outlook.

In contemporary applications such as representation learning, elements of a set (words, images, or signals) are mapped into high-dimensional vector spaces, where geometric relations encode similarity and structural information. These constructions provide concrete realizations of abstract linear-algebraic ideas.

Modern computational systems offer another illustrative example. Platforms devoted to *observability*—that is, the systematic measurement of the internal state of a running system—collect large families of numerical quantities such as latency, throughput, memory consumption, error rates, and resource utilization.

At a fixed instant of time, these measurements may be assembled into a vector

$$x(t) = (x_1(t), \dots, x_n(t)) \in \mathbb{R}^n,$$

whose coordinates represent observable characteristics of the system. As time evolves, the system determines a trajectory in this ambient linear space, so that monitoring and analysis may be interpreted geometrically as the study of paths in a finite-dimensional vector space.

Operational considerations typically impose bounds on each observable, expressed by inequalities of the form

$$a_i \leq x_i \leq b_i,$$

reflecting physical limitations or acceptable operating ranges. Geometrically, these constraints define a Cartesian product of intervals, namely an n -dimensional hyperrectangle (or, after normalization, a hypercube). Normal system behavior may therefore be viewed as confined to a bounded region of the ambient space, while anomalous behavior corresponds to trajectories departing from this region.

It should be emphasized that this construction is an idealized mathematical model: in practical systems the collection of observables may vary over time, so the dimension n is best regarded as fixed only locally or within a chosen representation. Nevertheless, the linear-space framework provides a natural language for describing states, constraints, and evolution.

In this way, observability systems illustrate a recurring theme of linear algebra: a flat ambient space serves as a universal stage upon which meaningful phenomena are characterized by geometrically defined subsets. Norm constraints give rise to spheres, anisotropic scalings produce ellipsoids, and coordinate-wise bounds define hyperrectangles. Despite their differing geometric appearances, all arise as structured subsets embedded within the same linear ambient space.

Historical remark (Angles and invariance).

Before eigenvectors were understood as invariant directions, their detection was often indirect. In two dimensions, determining whether a transformation preserves a direction reduces to detecting whether it preserves an angle.

This problem led naturally to the computation of arctangents. In the eighteenth century, J.H. Lambert introduced the continued fraction

$$\arctan(z) = \frac{z}{1 + \frac{z^2}{3 + \frac{4z^2}{5 + \frac{9z^2}{7 + \dots}}}},$$

as part of his work on orbital mechanics and rotational phenomena.

From the modern point of view, this analytic effort reflects a geometric question: does a linear transformation rotate a direction, or does it merely scale it? Eigenvectors provide the structural answer to this question, rendering angle computation auxiliary rather than fundamental.

Remark (Quadratic forms and the chi-square statistic). Expressions of the form defining ellipsoids appear naturally in contexts that are not, at first sight, geometric. A notable example arises in the theory of random number testing, where one encounters the quantity

$$V = \frac{(Y_1 - np_1)^2}{np_1} + \frac{(Y_2 - np_2)^2}{np_2} + \dots + \frac{(Y_n - np_n)^2}{np_n},$$

known in statistics as the *chi-square statistic* associated with the observed quantities $Y_1, \dots, Y_n$ and reference values $np_1, \dots, np_n$.

From the present point of view, this expression is simply a quadratic form on $\mathbb{R}^n$. It measures the squared length of the deviation vector

$$(Y_1 - np_1, \dots, Y_n - np_n)$$

with respect to a symmetric positive-definite bilinear form whose matrix, in the standard basis, is diagonal with entries $(np_1)^{-1}, \dots, (np_n)^{-1}$.

Consequently, the sets defined by inequalities of the form

$$V \leq c$$

are ellipsoids centered at the point $(np_1, \dots, np_n)$. The denominators np_i determine the anisotropic scaling along the coordinate directions, exactly as in the general ellipsoid

$$\sum_{i=1}^n \frac{(x_i - a_i)^2}{h_i^2} \leq r^2.$$

Thus, before any probabilistic interpretation is imposed, the chi-square expression is already a geometric object: it is the squared norm induced by a particular inner product on $\mathbb{R}^n$. Statistical theory makes use of this geometry, but does not create it.

Remark (The ambient vector space and constrained regions). The vector space E itself is the *ambient space* in which all vectors live. In coordinates, when a basis is chosen, this space appears as $\mathbb{R}^n$, a flat linear environment in which vectors may be freely added and scaled.

Geometric objects such as balls, spheres, and ellipsoids do not replace this ambient space; rather, they are subsets of it defined by constraints on the norm or on a quadratic form. For example, the sphere

$$S_r(x_0) = \{x \in E \mid \|x - x_0\| = r\}$$

selects vectors whose distance from x_0 is fixed, while an ellipsoid selects vectors whose coordinates satisfy an anisotropic quadratic inequality.

In many modern applications (AI Deep Learning for instance), algorithms operate on vectors in a high-dimensional ambient space while imposing such constraints during computation. Normalization steps may restrict vectors to spheres, regularization procedures may confine parameters to ellipsoidal regions, and optimization methods search for solutions within neighborhoods described by balls. The ambient vector space provides the linear universe in which vectors exist, while these geometric subsets describe regions selected by the rules or constraints of the problem.

Note (Discrete geometry: the Boolean hypercube).

Throughout this chapter, geometry has emerged from linear structure by equipping a vector space E over $\mathbb{R}$ with a bilinear form. The inner product gave rise to norms, distances, and topological notions such as open balls. Geometric objects—spheres, ellipsoids, hyperrectangles—appeared as subsets of the ambient linear space E selected by constraints on the norm or on a quadratic form.

This passage from algebra to geometry is not confined to the continuous setting. An analogous construction arises when the ambient space is replaced by a finite discrete set, producing a geometry that is combinatorial rather than metric in character.

The Boolean hypercube.

Let $n \geq 1$. The *Boolean hypercube* (or *n-dimensional hypercube*) is the set

$$Q_n = \{0, 1\}^n = \{(x_1, \dots, x_n) \mid x_i \in \{0, 1\}\}.$$

Its elements are called *Boolean vectors* or *binary strings of length n*.

The set Q_n contains 2^n elements, one for each way of assigning a binary value to each coordinate. These elements are called the *vertices* of the hypercube. Two vertices are declared *adjacent* if they differ in exactly one coordinate; such a pair is connected by an *edge*. The number of edges is $n \cdot 2^{n-1}$, since each of the 2^n vertices has exactly n neighbours.

This adjacency structure turns Q_n into a graph. More importantly, it equips Q_n with a canonical notion of distance: the *Hamming distance*

$$d_H(x, y) = \#\{i : x_i \neq y_i\},$$

which counts the number of coordinates in which two vertices differ. Two adjacent vertices are at Hamming distance 1.

The analogy with the continuous setting introduced in this chapter is precise. Just as the Euclidean norm on $\mathbb{R}^n$ is derived from the standard inner product and measures the “size” of a vector, the Hamming distance measures the “separation” of two binary configurations. And just as open balls in $\mathbb{R}^n$ are defined by the condition $\|x - x_0\| < r$, one defines Hamming balls

$$B_H(x_0, r) = \{y \in Q_n \mid d_H(x_0, y) \leq r\}$$

as subsets of Q_n centred at a vertex x_0 and of radius $r \in \{0, 1, \dots, n\}$.

The structure of Q_n generalises familiar geometric objects: for $n = 1$ the hypercube is a line segment, for $n = 2$ a square, and for $n = 3$ an ordinary cube, and so on.

Boolean functions and constrained subsets.

A *Boolean function* on n variables is a map

$$\Phi : \{0, 1\}^n \longrightarrow \{0, 1\}.$$

Given such a function, its *solution set* (or *satisfying set*) is

$$\mathcal{S}(\Phi) = \{x \in Q_n \mid \Phi(x) = 1\}.$$

This is precisely the subset of the hypercube selected by the constraint $\Phi(x) = 1$. The structural parallel with the continuous case is immediate: just as an ellipsoid is the subset of $\mathbb{R}^n$ selected by the constraint $B(x - x_0, x - x_0) \leq r^2$ for a symmetric positive-definite bilinear form B , the solution set $\mathcal{S}(\Phi)$ is the subset of Q_n selected by a Boolean constraint. In both cases, a function acting on the ambient space carves out a geometrically meaningful subset.

A particularly important class of Boolean functions arises in the context of *propositional satisfiability* (SAT [12]): the formula Φ is a conjunction of *clauses*, each of which is a disjunction of literals (variables or their negations). The question of whether $\mathcal{S}(\Phi) \neq \emptyset$ —that is, whether at least one satisfying assignment exists—is the *Boolean satisfiability problem*.

From the present geometric point of view, SAT asks whether a system of Boolean constraints leaves any vertex of Q_n standing. Each clause eliminates a subset of vertices: those assignments that violate it. The solution set is what remains after all eliminations. Proving unsatisfiability—that $\mathcal{S}(\Phi) = \emptyset$ —amounts to proving that every vertex of the hypercube is eliminated by at least one clause.

Relation to the linear framework.

The Boolean hypercube $\{0, 1\}^n$ can be embedded in $\mathbb{R}^n$ as a subset of the ambient vector space; its vertices are then points in $\mathbb{R}^n$ with coordinates in $\{0, 1\}$. However, the essential structure of Q_n is combinatorial and is not preserved by the linear operations of $\mathbb{R}^n$: the sum of two binary vectors is in general not a binary vector, and the Hamming distance does not arise from any inner product on $\mathbb{R}^n$.

In this sense, the Boolean hypercube illustrates a recurring theme: the ambient linear space $\mathbb{R}^n$ provides a universal stage, but the geometric objects of interest—whether ellipsoids defined by quadratic forms or solution sets defined by Boolean constraints—are subsets of that stage endowed with their own intrinsic structure. As noted in this chapter, such subsets are not vector spaces themselves; they are not closed under addition or scalar multiplication. Their geometry is determined by the constraints that define them, not by the linear structure of the ambient space.

Three related concepts arise naturally in this setting. The *search space* is the entire Boolean hypercube $Q_n = \{0, 1\}^n$: the universe of all configurations through which an algorithm may need to navigate. A procedure solving SAT has no a priori knowledge of where satisfying assignments are located; it only knows they must lie somewhere inside this exponentially large space. The *state space* is a closely related notion, slightly more general in scope: it refers to the set of all configurations or states the system may occupy, including, in more general algorithmic contexts, partial assignments or intermediate computational states. In the SAT setting, the state space and the search space coincide and are both identified with Q_n . The *solution space* is the subset

$$\mathcal{S}(\Phi) = \{x \in Q_n \mid \Phi(x) = 1\}$$

consisting precisely of the satisfying assignments. It is the geometrically meaningful object: the region of Q_n that survives after every clause has eliminated the vertices violating it.

A concrete example illustrates the elimination process. Consider the formula $\Phi(x_1, x_2) = x_1 \vee x_2$ on two variables. The search space is $Q_2 = \{00, 01, 10, 11\}$, a square with four vertices. Three assignments satisfy the formula:

$$\mathcal{S}(\Phi) = \{01, 10, 11\}.$$

The vertex 00 is the unique configuration eliminated by the clause $x_1 \vee x_2$. Even in this minimal example the central mechanism is visible: a Boolean constraint removes a subset of vertices from the hypercube, and the solution space is what remains.

Remark (Distribution of solutions). The Hamming metric allows one to ask how solutions are distributed within Q_n . Are the elements of $\mathcal{S}(\Phi)$ clustered in a small region of the hypercube, or are they spread throughout it? Can one pass from one solution to another by flipping a single bit, or are solutions separated by large Hamming distances? These questions, which are entirely natural from the geometric point of view, are central to the analysis of combinatorial optimization problems and to the theory of error-correcting codes, where Hamming balls play the role that Euclidean balls play in classical geometry.

Remark (Three instances of the same instinct). The Pythagorean theorem gave physical space a notion of distance by relating lengths through a quadratic identity. The inner product, introduced in this chapter, gave abstract vector spaces a notion of distance by specifying a symmetric positive-definite bilinear form. The Hamming metric gives the Boolean hypercube a notion of distance by counting coordinate disagreements. In each case, the procedure is the same: fix an ambient space, impose a rule that measures separation, and study the subsets selected by constraints relative to that rule.

Viewed from this perspective, a SAT solver is a procedure navigating a discrete geometric space of size 2^n , searching for vertices that survive a system of constraints.

Chapter 3

Tensors as Multilinear Maps

Linearity expresses that a map respects the algebraic structure of a vector space. Many constructions in mathematics, however, depend on functions that take several vector arguments simultaneously while remaining linear in each argument when the others are held fixed. This leads naturally to the notion of multilinearity.

Let $E_1, E_2, \dots, E_n$ and F be vector spaces over the same field K .

Definition 3.1 (Multilinear map). A map

$$T : E_1 \times E_2 \times \cdots \times E_n \rightarrow F$$

is called *multilinear* if it is linear in each argument separately. That is, for each index $i \in \{1, \dots, n\}$, and for all vectors

$$x_i, y_i \in E_i, \quad \alpha \in K,$$

we have

$$T(x_1, \dots, x_i +_{E_i} y_i, \dots, x_n) = T(x_1, \dots, x_i, \dots, x_n) +_F T(x_1, \dots, y_i, \dots, x_n),$$

and

$$T(x_1, \dots, \alpha \circ x_i, \dots, x_n) = \alpha \circ T(x_1, \dots, x_i, \dots, x_n),$$

with all other arguments held fixed.

When $n = 1$, a multilinear map is simply a linear map. Thus multilinear maps are not a new kind of object, but a direct generalization of linear transformations.

Multilinear maps arise naturally in many contexts: bilinear products, pairings between vectors and linear functionals, and higher-order interactions that cannot be reduced to a single input. Their systematic study leads to the concept of tensors.

Informally, a tensor is an object designed to encode multilinear behavior in an intrinsic and coordinate-free way.

There are two equivalent viewpoints that will be used throughout this document. The first treats tensors directly as multilinear maps.

Let E be a vector space over K , and let E^* denote its dual space, that is, the vector space of linear maps from E to K .

Definition 3.2 (Tensor as a multilinear map). A *tensor of type* (k, ℓ) on E is a multilinear map

$$T : \underbrace{E^* \times \cdots \times E^*}_{k \text{ times}} \times \underbrace{E \times \cdots \times E}_{\ell \text{ times}} \longrightarrow K.$$

Such a tensor takes k linear functionals and ℓ vectors as inputs, and produces a scalar. Linearity is required in each argument separately.

This definition makes precise several familiar cases:

- A scalar in K can be viewed as a tensor of type $(0, 0)$.
- A vector in E corresponds to a tensor of type $(0, 1)$.
- A linear functional in E^* is a tensor of type $(1, 0)$.
- A bilinear form on E is a tensor of type $(0, 2)$.
- If E is finite-dimensional, a linear map $T : E \rightarrow E$ can be identified with a tensor of type $(1, 1)$.

Thus vectors and matrices reappear as the lowest-order cases of a single unified framework.

At this level of abstraction, no coordinates, arrays, or geometric interpretations are involved. A tensor is defined entirely by how it acts on vectors and linear functionals, and by the multilinearity of this action.

The definition above is conceptually clear, but it has a practical limitation. Linear maps compose: if $S : E \rightarrow F$ and $T : F \rightarrow G$ are linear, then $T \circ S : E \rightarrow G$ is again linear, and its matrix is the product of the matrices of T and S . Multilinear maps do not admit an analogous operation. Their outputs and inputs do not naturally match in a way that allows canonical composition, and there is no general rule for combining two multilinear maps into a third while preserving multilinearity.

Multilinear maps do form vector spaces under pointwise operations, so they can be added and scaled. What is missing is a way to *reduce* multilinearity to linearity: to replace a multilinear problem by a linear one on a suitably constructed space. This is precisely what the tensor product achieves.

Let $E_1, \dots, E_n$ be vector spaces over the same field K . Consider the set of all multilinear maps

$$T : E_1 \times \cdots \times E_n \longrightarrow F$$

into an arbitrary vector space F . The tensor product is defined so that every such multilinear map corresponds uniquely to a linear map defined on a single vector space.

Definition 3.3 (Tensor product). The *tensor product* of the vector spaces $E_1, \dots, E_n$ is a vector space, denoted

$$E_1 \otimes \cdots \otimes E_n,$$

together with a multilinear map

$$\otimes : E_1 \times \cdots \times E_n \longrightarrow E_1 \otimes \cdots \otimes E_n,$$

satisfying the following universal property:

For every vector space F and every multilinear map

$$T : E_1 \times \cdots \times E_n \longrightarrow F,$$

there exists a unique linear map

$$\tilde{T} : E_1 \otimes \cdots \otimes E_n \longrightarrow F$$

such that

$$T(x_1, \dots, x_n) = \tilde{T}(x_1 \otimes \cdots \otimes x_n)$$

for all $x_i \in E_i$.

This property characterizes the tensor product completely. The elements $x_1 \otimes \cdots \otimes x_n$ are called *simple tensors*. Every element of the tensor product space is a finite linear combination of simple tensors.

As a consequence, the study of multilinear maps is equivalent to the study of linear maps defined on tensor products. Specifically,

$$\text{Multilinear maps } E_1 \times \cdots \times E_n \rightarrow F \quad \longleftrightarrow \quad \text{Linear maps } E_1 \otimes \cdots \otimes E_n \rightarrow F.$$

This equivalence is the conceptual foundation of tensor theory. It explains why tensors can be manipulated algebraically, composed with linear maps, and represented concretely once bases are chosen.

Returning to the case of a single vector space E , a tensor of type (k, ℓ) may equivalently be viewed as a linear map

$$T : \underbrace{E^* \otimes \cdots \otimes E^*}_k \otimes \underbrace{E \otimes \cdots \otimes E}_\ell \longrightarrow K.$$

Thus tensors are linear maps defined on tensor products, and their apparent complexity arises only when coordinates are introduced.

In contrast with the direct sum (Chapter I), which combines independent components additively, the tensor product encodes interactions between components through multilinearity.

Note: Once bases are chosen, tensor products become finite-dimensional vector spaces, and tensors admit coordinate representations as multidimensional arrays. These representations depend on the chosen bases and do not define the tensor itself.

Let E be a finite-dimensional vector space over K , and let

$$B = \{e_1, \dots, e_n\}$$

be a basis of E . Its dual basis

$$B^* = \{e^1, \dots, e^n\} \subset E^*$$

is defined by the relations

$$e^i(e_j) = \delta_j^i,$$

where δ_j^i is the Kronecker delta.

Every vector $x \in E$ admits a unique expansion

$$x = x^i \circ e_i,$$

and every linear functional $\varphi \in E^*$ admits a unique expansion

$$\varphi = \varphi_i \circ e^i.$$

The scalars x^i and φ_i are the *components* of x and φ relative to the chosen bases.

Let now T be a tensor of type (k, ℓ) on E , that is,

$$T : \underbrace{E^* \times \dots \times E^*}_k \times \underbrace{E \times \dots \times E}_\ell \longrightarrow K.$$

The action of T is completely determined by its values on basis elements:

$$T(e^{i_1}, \dots, e^{i_k}, e_{j_1}, \dots, e_{j_\ell}) = T^{i_1 \dots i_k}_{j_1 \dots j_\ell}.$$

These scalars are called the *components of T relative to the basis B* . They encode the tensor entirely once a basis has been fixed.

Given arbitrary arguments

$$(\varphi^1, \dots, \varphi^k, x_1, \dots, x_\ell),$$

multilinearity implies the expansion

$$T(\varphi^1, \dots, \varphi^k, x_1, \dots, x_\ell) = T^{i_1 \dots i_k}_{j_1 \dots j_\ell} \varphi_{i_1}^1 \dots \varphi_{i_k}^k x_1^{j_1} \dots x_\ell^{j_\ell}.$$

Thus index notation is the coordinate expression of multilinearity after a basis is chosen.

Bilinear maps provide the simplest computational example of multilinearity. A bilinear map

$$B : V \times W \rightarrow U$$

is linear in each argument separately. Once bases are chosen in the vector spaces involved, the map is represented by coefficients t_{ij}^k such that the components of the output vector satisfy

$$z^k = \sum_{i=1}^m \sum_{j=1}^n t_{ij}^k x^i y^j. \quad (3.1)$$

The collection of coefficients (t_{ij}^k) forms a rank-three tensor. This representation shows that familiar computational procedures are coordinate expressions of multilinear mappings. For example, the multiplication of complex numbers

$$(x_1 + ix_2)(y_1 + iy_2) = (x_1y_1 - x_2y_2) + i(x_1y_2 + x_2y_1) \quad (3.2)$$

can be interpreted as the evaluation of two bilinear forms determined by a $2 \times 2 \times 2$ tensor of coefficients.

From this viewpoint, algorithms manipulating numerical arrays are implementations of tensorial transformations written in a chosen reference system. Computation therefore makes explicit the invariant multilinear structure underlying algebraic operations.

In particular, a bilinear map

$$B : E \times F \rightarrow G$$

between finite-dimensional vector spaces is represented, once bases are chosen, by components

$$B_{ij}^k,$$

so that for vectors $x \in E$ and $y \in F$,

$$(B(x, y))^k = B_{ij}^k x^i y^j.$$

Thus multidimensional arrays of coefficients appearing in numerical algorithms are coordinate representations of tensors encoding multilinear operations.

Once a basis has been fixed, the components

$$T^{i_1 \dots i_k}_{j_1 \dots j_\ell}$$

can be arranged in a multidimensional array of scalars.

Changing the basis of E changes the numerical values of the components according to precise transformation rules, while the tensor itself remains unchanged.

Note: Tensors via Transformation Laws

The definition of a tensor as a multilinear map is intrinsic and coordinate-free. An equivalent characterization emphasizes how the components of a tensor transform under a change of basis.

This viewpoint makes explicit the invariant content that survives changes of coordinates, which is precisely what allows tensorial laws to appear in physics and in engineering applications independent of the chosen representation.

Let $B = \{e_i\}$ and $B' = \{e'_i\}$ be two bases of E , related by

$$e'_i = A^j_i e_j,$$

where A^j_i is an invertible change-of-basis matrix. The corresponding dual bases satisfy

$$e'^i = (A^{-1})^i_j e^j.$$

Let T be a tensor of type (k, ℓ) on E , with components relative to B given by

$$T^{i_1 \dots i_k}_{j_1 \dots j_\ell}.$$

The matrix A^j_i expresses the new basis vectors in terms of the old ones, as discussed in Chapter I. Accordingly, the transformation rule below describes how the coordinate representation of T changes when passing from the basis B to B' .

The components relative to the new basis B' are given by

$$T'^{i_1 \dots i_k}_{j_1 \dots j_\ell} = A^{i_1}_{p_1} \dots A^{i_k}_{p_k} (A^{-1})^{q_1}_{j_1} \dots (A^{-1})^{q_\ell}_{j_\ell} T^{p_1 \dots p_k}_{q_1 \dots q_\ell}.$$

Contravariant indices (upper indices) transform with the matrix A , while covariant indices (lower indices) transform with its inverse. This transformation rule is not an additional assumption. It is a direct consequence of multilinearity and of how vectors and dual vectors transform under a change of basis.

Conversely, suppose that for each choice of basis one assigns a multidimensional array of scalars

$$T^{i_1 \dots i_k}_{j_1 \dots j_\ell}$$

in such a way that these arrays are related by the transformation law above whenever the basis is changed. Then these coordinate collections define a unique multilinear map

$$T : \underbrace{E^* \times \dots \times E^*}_k \times \underbrace{E \times \dots \times E}_\ell \longrightarrow K,$$

independent of the chosen basis.

Thus the following two viewpoints are equivalent:

- A tensor is a multilinear map.
- A tensor is a collection of components that transform according to the tensor transformation law.

The first definition emphasizes intrinsic structure. The second emphasizes invariance under change of reference frame. Their equivalence explains why tensor theory plays a central role in geometry and physics: tensorial equations remain valid in every coordinate system, because both sides transform according to the same law.

Thus:

- tensors are abstract multilinear objects;
- arrays are basis-dependent representations;
- indices record how multilinearity decomposes into coordinates.

This distinction is essential for interpreting computational implementations.

Up to this point, tensors have been considered as static multilinear objects. We now introduce the operation that allows them to interact algebraically.

To combine tensors and produce new multilinear objects, one introduces the fundamental operation of tensor contraction, which generalizes matrix multiplication and explains the appearance of summation over repeated indices.

We denote by

$$\mathcal{T}^{(k,\ell)}(E)$$

the vector space of all tensors of type (k, ℓ) on E .

Let E be a finite-dimensional vector space over K . Consider a tensor

$$T \in \mathcal{T}^{(k,\ell)}(E)$$

and a tensor

$$S \in \mathcal{T}^{(m,n)}(E).$$

Assume that one covariant slot of T and one contravariant slot of S are selected. Tensor contraction is the operation that pairs these two slots using the natural evaluation map

$$E^* \times E \rightarrow K.$$

Definition 3.4 (Tensor contraction). Let

$$T \in \mathcal{T}^{(k,\ell)}(E), \quad S \in \mathcal{T}^{(m,n)}(E).$$

Choosing one covariant slot of T and one contravariant slot of S , their contraction is defined by composing these arguments with the natural evaluation pairing

$$\langle \cdot, \cdot \rangle : E^* \times E \rightarrow K.$$

In coordinates, this operation appears as summation over a basis.

The resulting tensor has type

$$(k + m - 1, \ell + n - 1).$$

Contraction reduces the total number of tensor slots by two: one contravariant and one covariant slot are paired and eliminated. No coordinates are required to define this operation.

To express contraction in coordinates, a basis must be chosen. The abstract pairing then appears as summation over repeated indices.

Fix a basis $\{e_i\}$ of E and its dual basis $\{e^i\}$.

Let

$$T^i_j \quad \text{and} \quad S^j_k$$

be the components of two tensors of type $(1, 1)$.

The contraction over the index j produces a new tensor with components

$$R^i{}_k = T^i{}_j S^j{}_k.$$

Here the repeated index j indicates summation:

$$R^i{}_k = \sum_{j=1}^n T^i{}_j S^j{}_k.$$

This summation is not an additional rule. It is the coordinate expression of the abstract pairing between E^* and E .

Let $A : E \rightarrow E$ and $B : E \rightarrow E$ be linear maps. Each can be identified with a tensor of type $(1, 1)$. Relative to a basis, their components are written as

$$A^i{}_j, \quad B^j{}_k.$$

The composition $A \circ B : E \rightarrow E$ has components

$$(A \circ B)^i{}_k = A^i{}_j B^j{}_k.$$

Thus matrix multiplication is exactly tensor contraction between two $(1, 1)$ tensors, and the familiar summation rule for matrix products is a special case of the abstract pairing between E^* and E .

At the abstract level we have:

Tensor contraction

which, after choosing bases, becomes

$$\text{Index summation} \quad \longrightarrow \quad C^i{}_k = \sum_j A^i{}_j B^j{}_k.$$

In computational and modeling contexts, tensors frequently appear through multidimensional arrays whose indices label independent parameters of a problem. The following example illustrates how such arrays arise in practice while still representing tensorial data.

Consider a physical system modeled on a discrete two-dimensional grid. At each spatial point (i, j) and at each time step t , a force vector

$$F(t, i, j) \in \mathbb{R}^2$$

is measured, with components (F_x, F_y) relative to a fixed spatial frame.

Let:

- $t \in \{1, \dots, T\}$ index time,
- $(i, j) \in \{1, \dots, N\} \times \{1, \dots, M\}$ index spatial position,

- $\alpha \in \{1, 2\}$ index force components.

The collected data can be written as an array

$$F_{tij\alpha},$$

with shape $(T, N, M, 2)$.

Each index corresponds to an independent modeling choice: time, space, and vector components. The array stores numerical values relative to chosen coordinate systems. Only the component index α corresponds to a genuine vector slot in the tensorial sense; the indices (t, i, j) label independent parameters of the model. Contracting the index α against a linear functional produces, at each spacetime point, a scalar measurement of the force along a chosen direction.

Abstractly, this data can be interpreted as the coordinate representation of a tensor-valued function on a discrete domain.

Note: Linear Algebra Realized in Hardware — Systolic Arrays [16]

The interpretation of matrix multiplication as tensor contraction is not only a conceptual reformulation. It also determines how modern computational hardware is designed.

In large numerical computations, the dominant operation is the repeated evaluation of inner products,

$$c_i = \sum_{j=1}^n a_{ij} b_j,$$

which is the coordinate expression of a linear map applied to a vector. Equivalently, it is a contraction between a tensor of type $(1, 1)$ and a tensor of type $(0, 1)$.

In the late 1970s, H. T. Kung and C. E. Leiserson proposed organizing computation around this operation through architectures called *systolic arrays*. Instead of moving data back and forth between memory and a central processor, data flows rhythmically through a grid of simple processing elements. Each element repeatedly performs a single operation of the form

$$C \leftarrow AB + C,$$

a multiply-accumulate step implementing one contribution to an inner product.

Mathematically, each processing element evaluates a partial tensor contraction. As data propagates across the array, successive contractions accumulate, and the global result emerges from the coordinated action of many local linear operations.

This organization reflects a structural principle: hardware efficiency is achieved when computation mirrors algebraic structure. Since linear transformations decompose into sums of products, an architecture optimized for multiply-accumulate operations naturally implements linear algebra at scale.

Modern high-performance computing architectures frequently adopt variants of this paradigm, arranging many simple processing elements to evaluate multiply-accumulate operations in parallel. What appears at the software level as tensor contraction or matrix multiplication is therefore executed physically as streams of coordinated inner products.

From the present viewpoint, this development is not accidental. Tensor contraction is the fundamental interaction between multilinear objects, and large-scale computation reduces complex algorithms to repeated evaluations of this interaction.

In this sense, numerical arrays stored in memory, index summations written in formulas, and data flowing through silicon circuits are three realizations of the same mathematical object: the evaluation of multilinear maps.

The abstract principles developed above appear concretely in classical physics, where tensorial quantities describe invariant relations between physical observables.

A classical example: the inertia tensor [13]

In classical mechanics, the distribution of mass in a rigid body is encoded by a second-order tensor called the *inertia tensor*. Relative to a chosen orthonormal basis of $\mathbb{R}^3$, its components are given by

$$I_{ij} = \int_{\Omega} (\delta_{ij} \|x\|^2 - x_i x_j) dm,$$

where Ω is the body and x is the position vector.

The inertia tensor defines a linear relation between angular velocity ω and angular momentum L :

$$L_i = I_{ij} \omega^j.$$

Under a change of basis determined by an invertible matrix A , its components transform according to

$$I'_{ij} = A^p_i A^q_j I_{pq},$$

which is precisely the transformation law of a tensor of type $(0, 2)$.

Thus, although the numerical entries of I_{ij} depend on the chosen reference frame, the geometric object it represents does not. The physical content—resistance to rotational acceleration—is invariant under change of coordinates.

Tensor theory therefore extends the central principle introduced in Chapter I: mathematical structure is identified not by its numerical representation, but by the invariance of its defining relations under admissible transformations.

Notes and Remarks

Outlook: Computation Beyond Electronics [17]

Recent developments in computing architectures explore physical substrates beyond traditional electronic circuits, including optical and photonic systems in which information is processed through interference, diffraction, and propagation of light.

Despite their physical differences, such systems are designed to implement the same fundamental algebraic operations encountered throughout this chapter. In particular, the dominant computational task remains the evaluation of expressions of the form

$$C^i_k = A^i_j B^j_k,$$

that is, tensor contraction.

In these emerging architectures, the interaction of physical signals naturally realizes multiply-accumulate operations in parallel, so that the evaluation of multilinear maps is performed directly by the underlying physical process rather than by sequential electronic instructions.

A non-trivial constraint, however, arises from the physical encoding of information. Non-coherent optical systems represent values as light intensities, which are intrinsically non-negative; they therefore operate within the restricted domain $\mathbb{R}^+$ rather than the full real line. Since tensor contractions require weights and activations that range over all of $\mathbb{R}$, such systems are algebraically incomplete for general computation. Coherent architectures address this limitation by encoding values as optical field *amplitudes*—quantities that carry sign—and recovering the signed output via interference at detection. The full group $(\mathbb{R}, +)$ is thereby made available, and tensor contractions of arbitrary sign are realizable in hardware.

From this perspective, advances in hardware do not alter the mathematical structure of computation. They provide new physical realizations of the same invariant operations: linear transformations and tensor contractions, now extended to their natural domain. The algebraic completeness required here—the passage from $\mathbb{R}^+$ to $\mathbb{R}$ —is precisely the structural distinction between a multiplicative and an additive group, whose correspondence is the subject of Chapter IV.

Thus, independently of whether computation is carried out by electronic, optical, or other physical means, the algebraic framework developed here remains unchanged.

Remark: Bridge to Chapter II — the chi-square statistic as a $(0, 2)$ tensor

The chi-square statistic encountered in Chapter II provides an earlier example of a tensor of type $(0, 2)$. Recall that the expression

$$V = \sum_{i=1}^n \frac{(Y_i - np_i)^2}{np_i}$$

was identified as the squared norm induced by a symmetric positive-definite bilinear form B on $\mathbb{R}^n$, whose matrix in the standard basis is diagonal with entries $(np_i)^{-1}$. From the present point of view, B is precisely a tensor of type $(0, 2)$: a multilinear map

$$B : \mathbb{R}^n \times \mathbb{R}^n \longrightarrow \mathbb{R}$$

that is symmetric and positive-definite. Its coordinate representation changes with the basis, but the geometric object it defines—the ellipsoidal level sets of V —does not. Thus the statistical quantity V , the quadratic form of Chapter II, and the $(0, 2)$ tensor of the present chapter are three descriptions of the same invariant structure.

Note: OLAP Cubes and Tensor Contraction

In large-scale data analysis, observations are frequently organized along multiple independent categorical dimensions such as time, location, system component, or measurement type. The resulting structure is commonly called an *OLAP cube* (Online Analytical Processing cube).

Mathematically, such an object assigns numerical values to elements of a Cartesian product

$$D_1 \times D_2 \times \cdots \times D_n,$$

where each set D_i represents one observational dimension. After enumerating these dimensions, the data can be written as a multidimensional array

$$F_{i_1 i_2 \dots i_n},$$

which is precisely the coordinate representation of a tensor indexed by n independent parameters.

In practice, only a small subset of all possible index combinations is observed, so the array is typically sparse. This again reflects a distinction emphasized throughout this chapter: arrays depend on chosen coordinates and storage conventions, while the underlying multilinear structure is independent of representation.

Analytical queries often aggregate data along selected dimensions. For instance, combining observations while ignoring certain indices produces quantities of the form

$$G_{i_1 \dots i_k} = \sum_{j_1, \dots, j_m} F_{i_1 \dots i_k j_1 \dots j_m}.$$

From the tensorial viewpoint, this operation is a contraction: indices corresponding to aggregated dimensions are summed over, reducing the order of the tensor.

Thus operations known operationally as filtering, grouping, or aggregation correspond mathematically to projections of a high-dimensional tensor onto lower-dimensional subtensors. The OLAP cube therefore provides a concrete realization of tensorial structure arising in modern data analysis.

This example reinforces a recurring theme: tensor contraction appears wherever multidimensional relations are summarized through systematic aggregation.

Chapter 4

Logarithms as Structure-Preserving Transformations

In the preceding chapters, invariance under change of coordinates was identified as the defining characteristic of geometric and tensorial objects. A vector, a bilinear form, a tensor — each was defined not by its numerical components, but by the structural relations it preserves under transformation. The present chapter observes that this same principle operates outside the linear setting. The logarithm is an isomorphism between algebraic structures, converting multiplication into addition while preserving every relation that matters. The invariant is not a coordinate, a length, or a tensor component — it is the algebraic correspondence itself.

At the most basic level, a logarithm answers the question: *what exponent produces a given number?* Yet historically and mathematically, logarithms owe their power to something deeper: they convert multiplication into addition — and they do so exactly.

In Chapter I, an isomorphism was defined as a structure-preserving mapping between algebraic systems. The logarithm provides a precise, global realization of that idea.

The mapping

$$\log : \mathbb{R}^+ \rightarrow \mathbb{R}$$

is not merely a bijection; it is an isomorphism between two group structures:

1. A multiplicative group $(\mathbb{R}^+, \cdot)$.
2. An additive group $(\mathbb{R}, +)$.
3. An isomorphism

$$\log : (\mathbb{R}^+, \cdot) \longrightarrow (\mathbb{R}, +),$$

whose inverse is $\exp : (\mathbb{R}, +) \rightarrow (\mathbb{R}^+, \cdot)$.

The defining property is:

$$\log(ab) = \log(a) + \log(b), \quad \log(1) = 0.$$

Multiplication corresponds to addition; multiplicative identity corresponds to additive identity; multiplicative inverses correspond to additive inverses. No algebraic relation is lost — only its appearance changes.

This is not a numerical trick or a computational shortcut. It is a structural equivalence: the two groups are, from an algebraic standpoint, the same object wearing different notation.

This isomorphism is made physical in the slide rule. Distances on a logarithmic scale represent logarithmic values. Adding distances corresponds to multiplying numbers.

The construction of logarithmic scales proceeds by mapping

$$x \mapsto \log_{10}(x),$$

placing numbers at distances proportional to their logarithms. Equal ratios correspond to equal distances. The structural identity $\log(ab) = \log(a) + \log(b)$ becomes a mechanical operation: slide one rule against another, and multiplication is performed by addition of lengths.

Lengths are not numbers, but they obey the same structural rules. By exploiting this correspondence, the slide rule becomes an analog computer — a physical model of an algebraic isomorphism.

Logarithms also admit a purely geometric definition independent of exponentiation. By construction,

$$\ln(a) = \int_1^a \frac{1}{x} dx,$$

the signed area under the curve $y = \frac{1}{x}$ from 1 to a .

This integral definition makes the isomorphism property transparent. For $a, b > 0$,

$$\ln(ab) = \int_1^{ab} \frac{1}{x} dx = \int_1^a \frac{1}{x} dx + \int_a^{ab} \frac{1}{x} dx.$$

The substitution $x = at$ in the second integral gives

$$\int_a^{ab} \frac{1}{x} dx = \int_1^b \frac{1}{t} dt = \ln(b),$$

and therefore $\ln(ab) = \ln(a) + \ln(b)$. The group isomorphism is a theorem, not an assumption.

The integral definition also yields the fundamental derivative formula immediately:

$$\frac{d}{da} \ln(a) = \frac{1}{a}.$$

The behavior of $\frac{d}{da} \ln(a) = \frac{1}{a}$ connects the logarithm to the general framework of derivatives and local linearization.

Recall that for a function $y = f(x)$, the derivative at a point a is

$$f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h},$$

the instantaneous rate of change, and geometrically the slope of the tangent line. Near a , a differentiable function behaves approximately like its tangent line:

$$f(x) \approx f(a) + f'(a)(x - a).$$

This local linearity mirrors the passage from tensors to their components discussed in Chapter II: global nonlinear structure becomes tractable when approximated by its best linear model at each point. The derivative replaces a nonlinear map by a linear one locally; the logarithm replaces a multiplicative structure by an additive one globally. Both are instances of the same organizing principle.

The derivative of the logarithm also enables its efficient numerical computation via the Newton–Raphson method. Suppose we wish to find $\ln(x)$, i.e., to solve the equation

$$e^y = x,$$

or equivalently $f(y) = e^y - x = 0$.

The Newton–Raphson iteration replaces the nonlinear equation by a sequence of linear ones. Starting from a guess y_n , we approximate f by its tangent line and solve:

$$0 = f(y_n) + f'(y_n)(y_{n+1} - y_n).$$

Since $f'(y) = e^y$, this yields

$$y_{n+1} = y_n - \frac{e^{y_n} - x}{e^{y_n}} = y_n + \frac{x - e^{y_n}}{e^{y_n}}.$$

Starting from $y_0 = 1.5$, successive iterations converge rapidly to $\ln(5) \approx 1.60944$.

The simplicity of this iteration is not accidental. Working in base e ensures that $\frac{d}{dy}e^y = e^y$, minimizing algebraic overhead and improving numerical stability. Logarithms in other bases are recovered by the change-of-base formula:

$$\log_b(x) = \frac{\ln(x)}{\ln(b)}.$$

The same structural idea underlies historical algorithms for constructing logarithm tables. Henry Briggs computed base-10 tables by accumulating small multiplicative corrections; a closely related method suited to binary arithmetic is described in [9].

The algorithm builds a target number by successive multiplications of the form

$$\frac{10^k}{10^k - 1},$$

accumulating the corresponding logarithms whenever the partial product does not overshoot the target. What appears as computation is, in fact, structure preservation: each multiplicative step corresponds to an additive increment on the logarithmic side, faithfully enacting the isomorphism $\log(ab) = \log(a) + \log(b)$.

The natural base e is not an arbitrary choice. It is the unique base for which $\frac{d}{dx}e^x = e^x$, and the unique solution to $\int_1^e \frac{1}{x} dx = 1$. Its arithmetic character is revealed by several independent constructions.

One classical definition arises from compound interest:

$$e = \lim_{n \rightarrow \infty} \left(1 + \frac{1}{n}\right)^n.$$

Another is provided by its continued fraction expansion:

$$e = [2; 1, 2, 1, 1, 4, 1, 1, 6, \dots],$$

written in expanded form as

$$e = 2 + \frac{1}{1 + \frac{1}{2 + \frac{1}{1 + \frac{1}{1 + \frac{1}{4 + \dots}}}}}$$

These representations converge rapidly and are well suited for computation, but their deeper significance is structural: e is the natural unit of the isomorphism between $(\mathbb{R}^+, \cdot)$ and $(\mathbb{R}, +)$.

The logarithm transforms nonlinear growth into linear motion. It converts multiplication into addition, products into sums, and exponential scales into linear ones.

Two complementary manifestations of the invariant spirit are therefore visible. The logarithm is a *global* isomorphism: it establishes a perfect structural equivalence between two algebraic worlds, valid everywhere on $\mathbb{R}^+$. The derivative, by contrast, provides a *local* linearization: it approximates an arbitrary transformation by its best linear model at each point.

In both cases, complexity is rendered intelligible by expressing it through linear relations. This is why logarithms appear simultaneously in analysis, geometry, numerical methods, and physical instruments. They do not belong to a single domain. They preserve structure across domains.

Bibliography

- [1] Iribarren Ignacio L., *Álgebra Lineal*, Editorial Equinoccio, 2009.
- [2] Halmos, Paul R., *Finite-Dimensional Vector Spaces*, Springer-Verlag, 1987.
- [3] Iribarren, Ignacio L., *Topología de Espacios Métricos*, Editorial Limusa-Wiley, 1973.
- [4] Cohen, Leon and Ehrlich, Gertrude, *The Structure of the Real Number System*, D. Van Nostrand Company, 1963.
- [5] Wilder, Raymond L., *Introduction to The Foundations of Mathematics*, Dover Publications, Inc., 2012.
- [6] Greub, Werner, *Linear Algebra*, Springer-Verlag, 1975.
- [7] Lukas, Andre, *The Oxford Linear Algebra for Scientists*, Oxford University Press, 2022.
- [8] Apostol, Tom M., *Calculus Volume I*, John Wiley & Sons Inc., 1967.
- [9] Knuth, Donald E., *The Art of Computer Programming Volume 1: Fundamental Algorithms*, Addison-Wesley, 1997.
- [10] Knuth, Donald E., *The Art of Computer Programming Volume 2: Seminumerical Algorithms*, Addison-Wesley, 1981.
- [11] Knuth, Donald E., *The Art of Computer Programming Volume I, Fascicle 1: MMIX—A RISC Computer for the New Millennium*, Addison-Wesley, 2005.
- [12] Knuth, Donald E., *The Art of Computer Programming Volume 4B: Combinatorial Algorithms*, Pearson Education, Inc., 2023.
- [13] Feynman Richard P., Leighton Robert, Sands Matthew, *The Feynman Lectures on Physics, Volume II*, California Institute of Technology, 2010.
- [14] Kolecki, Joseph C., *An Introduction to Tensors for Students of Physics and Engineering*, NASA/TM-2002-211716, 2002.
- [15] Kolecki, Joseph C., *Foundations of Tensor Analysis for Students of Physics and Engineering With an Introduction to the Theory of Relativity*, NASA/TP—2005-213115, 2005.
- [16] Kung, H. T., Leiserson, Charles L., *Introduction to VLSI Systems - Algorithms for VLSI Processor Arrays*, ed. C. Mead and L. Conway, Addison-Wesley, Reading, MA, 1980.

- [17] Shaofu Xu, Jing Wang, Haowen Shu, Zhike Zhang, Sicheng Yi, Bowen Bai, Xingjun Wang, Jianguo Liu, Weiwen Zou, *Optical coherent dot-product chip for sophisticated deep learning regression*, Nature — Light: Science & Applications, 2021.